

MERCEDES-BENZ · AIOS · MVA

Design Sync

MVA — product strategy, the assistant's form, and generative UI for I3 MAX.

PRODUCT × DESIGN • I3 MAX • FOR REVIEW

SCROLL

01

Overall MVA product strategy

MVA is evolving from a voice assistant into the intelligence layer of the whole cabin — it holds a conversation, gets things done, remembers, and coordinates across apps. Seven directions:

01 Natural full-duplex dialogue

LLM-driven and genuinely human to talk to — it holds more context, sounds more natural, and won't fire when you didn't mean it.

02 Collaborative task completion

You ask in plain language, and it breaks the request into steps and carries them out itself — finding an app, installing it, opening it, and taking you straight to the right page.

03 Cross-app orchestration

It reaches across app boundaries, combining the cabin's building-block capabilities to bring you the service or content directly — instead of making you hop from app to app.

04 Memory & personalization

It remembers your habits and preferences, both short-term and long-term, and reaches out at the right moment — timed from real usage, not guesswork.

05

Adaptive form

It takes different forms across screens and hardware, reshaping to the moment and generating the content that fits it.

06

Multimodal perception

It senses the world and reflects it back — sight, text, sound, and the state of the cabin, all fused through the model — for more direct feedback and real mutual trust.

07

Expert orchestration

It marshals specialist expert agents, hands each the work it does best, and turns their expertise into one seamless conversation with you.

Direction 5 — adaptive form — leads straight to a design question: what should all that intelligence actually look like on screen?

↓ TOPIC 02

02

The MVA “avatar” for I3 MAX

As the face of AIOS, MVA should be an **abstract form — like Siri and Gemini** — not a solid face or character. Our current Starlight (星辉) and little-Benz (小奔) are still too literal; for I3 MAX we want to move beyond them to something fully abstract. Whether that turns out to be light, an orb, or something else is still open — the point is that it's abstract, not a character.

Starting from I2 MAX, we want the whole system to re-skin for holidays and campaigns, Tesla-style — the avatar included. And once you commit to theming, the form has to stay abstract to stay flexible. That's one of the biggest reasons to keep it abstract instead of a fixed character.

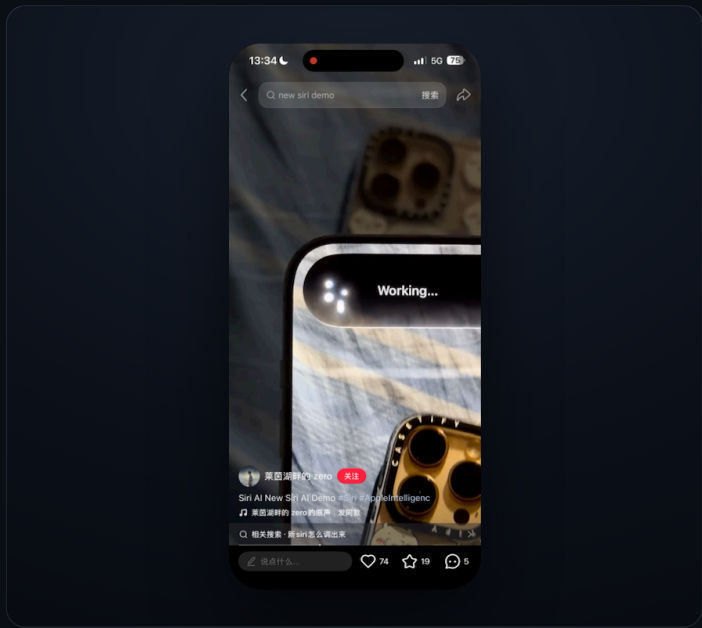
Parametric, not an asset. An abstract form is really just a set of parameters — color, motion, density — so a new theme is just a new preset. A character is a fixed asset: you can put it in a costume, but it still reads as the same character, never as the theme itself.

The identity paradox. A character's whole value is a stable, recognizable identity — but theming demands change, so the more you alter it, the more you erode the very thing that made it worth having. An abstract form has no fixed identity to erode, so it re-skins endlessly. That's exactly what our planned celebrity personas need: a base you can dress in a star's voice and visual signature, tailor to each user, and retire whenever you want. A fixed character can only ever be itself; an abstract presence can put on any star's cues — their signature, palette, and motion — say 张艺兴, 苏翊鸣, 王楚钦.

Coherence & cadence. An abstract form is made of the system's own material, so it re-skins in step with everything around it — a dozen fresh looks a year. A character is a foreign object: it has to be hand-matched to each theme, always looks a little bolted-on, and you'll manage two a year at best.

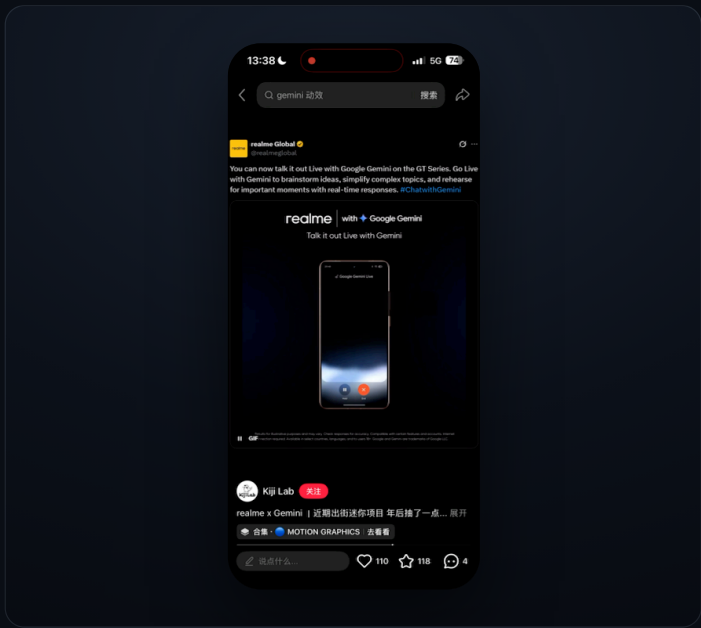
The industry already turned. Tesla, the company most committed to theming its whole system, has no avatar at all — its personality lives entirely in motion, sound, and light. And Siri, Gemini, ChatGPT, and Super Eva are all moving their assistants toward an abstract form too.

◆ WHAT “ABSTRACT” LOOKS LIKE IN THE WILD



Apple Intelligence · Siri

the screen-edge glow, in use



Google · Gemini

Gemini Live, in use

Both assistants in use — presence expressed as light and gradient, not a character.

◆ SUPPORTING REASONS

Positioning. An abstract presence feels like the whole system is answering you — exactly the level an OS assistant should operate at. A character shrinks that down to “a thing that lives inside the screen.”

It fits any screen. We design across many sizes, small screens included — and a solid avatar needs real width and a fixed spot to sit on every one of them. An abstract presence has no footprint to lay out; it just melts into the UI that's already there. Across every size, that's one less thing for design to solve — and less to carry.

Warmth through behavior, not a face. The personality comes through in how it breathes, anticipates, and responds — in its motion, its voice, its tone.

◆ WE'VE ALREADY SOLVED OVERALL MVA PRESENTATION

The Nebula window and the smart island beneath it already act as the AI's body and status display — they carry the activity, the results, and the state. Put a heavy, shape-shifting character on top of that and you're paying for the same job twice, with two things fighting for attention. The cleaner split: let the window and island carry the **what**, and let MVA just signal that it's **alive and listening** — one quiet, ambient cue you catch out of the corner of your eye, with nothing extra pulling the driver's focus.

Restraint here isn't compromise, it's maturity.

■ ALL IN ALL — WHAT WE NEED FOR I3 MAX

1 An abstract avatar + Nebula window + smart island, designed as one

smart island = the former smart bar

- One shared style and ambient-effect system across all three.
- Their motion stays cohesive across all three — for example:
 - the avatar's expression reflects what's showing in the Nebula window;
 - a recommendation surfaces in the smart island, then settles into a card in the feed.

2 MVA lives inside the overall system design — not beside it or added on to it

- **Onboarding** — the avatar is present from first run.
- **Profile** — customization ties to each user's profile — every user's MVA is their own, with an MVA “memory board” living in the profile module.
- **Ambient language** — the ways MVA expresses itself across modules:
 - powering up any module (for example, image optimization);
 - execution — a “say what you see” animation as it acts;
 - AI recommendation.

That ambient language is really Gen-UI applied across the system — and that's where we'd most like your input.

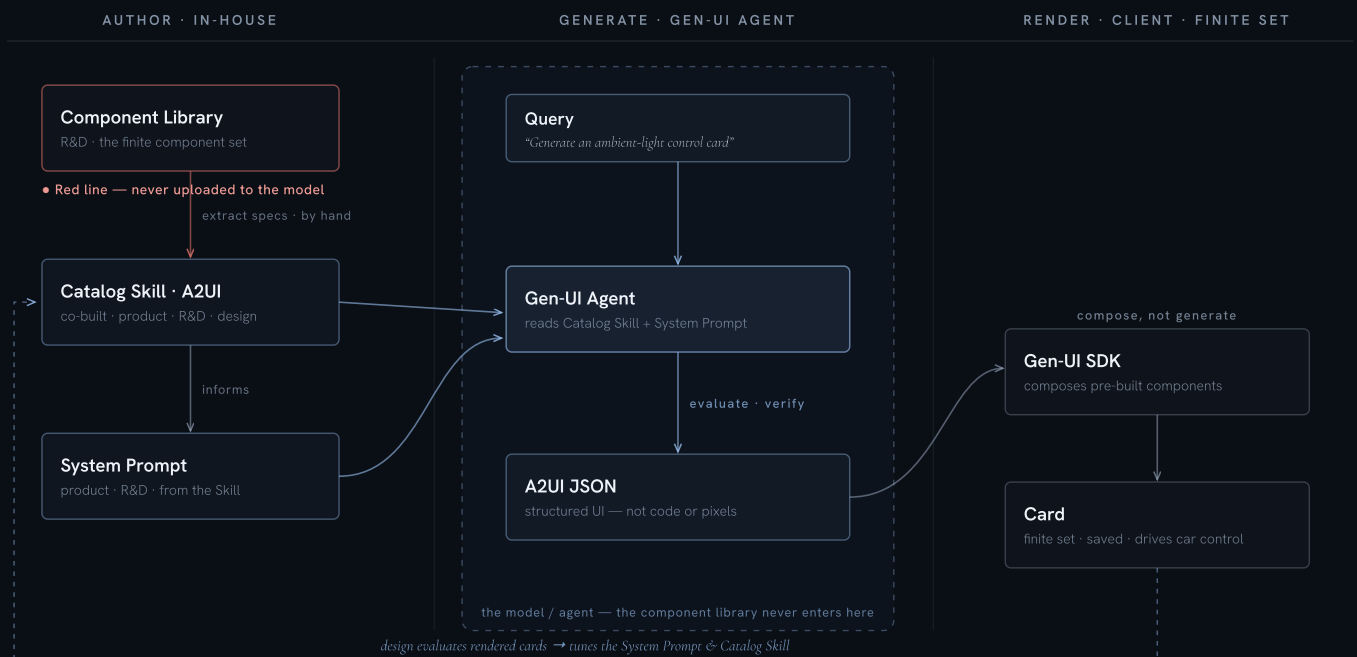
That ambient language is really Gen-UI applied across the system. If we do generate components, here's the pipeline — and where design sits in it.

↓ TOPIC 03

03

Gen-UI & widgets for I3 MAX

Whatever design decides, the bar is the same: across two surfaces — **chat** UI and the **vehicle-control** components — we need to meet very different, scene-by-scene visual needs as flexibly as we can (the full scenario list is tracked separately). And if design does choose to **generate** some of those components with Gen-UI, here's how the flow works — and the two places where design holds the wheel.



The pipeline end to end: authored in-house, generated as structured A2UI JSON, then composed from a fixed set of components on the client — with design closing the loop.

RED LINE *The component library never goes to the model.* It's built for R&D — the finite set of real components — and it never gets uploaded to the model. That's the hard line. The model only ever sees the Catalog Skill and the System Prompt.

The Catalog Skill is the quality gate. A document that product, R&D, and design build together, in Google's A2UI format — it lifts the component library's design specs and guardrails into a per-component schema the model can actually read. Generation quality lives or dies here.

The System Prompt is the second lever. Product and R&D write it from the Skill, and it further shapes what the model is allowed to produce.

The SDK composes, it doesn't generate. R&D's Gen-UI SDK assembles pre-built components for each scene from a fixed set — it composes, it doesn't invent free-form front-end. And because that set is bounded, design can review every result and feed corrections back into the prompt and the Catalog Skill.

Design holds the wheel at both ends — writing the Catalog Skill that bounds what can be built, and judging the cards that tune it. Nothing is generated live on the device: the cards are composed from pre-built components, and their JSON is generated and saved behind the scenes — so every step of that pre-build stays bounded by design.

NEXT STEP — THE ASK

Product, R&D, and design run this pipeline together — one joint pass, end to end.

- 01 Is the **flow actually feasible** in practice — does it hold together end to end?
- 02 Do the **generated results clear design's quality bar**?



MERCEDES-BENZ